Name : Postiwala Vatsal PRN No: 26070122509 Batch : B3(2025-2029)

ASSIGNMENT No: 14

TITLE: Write a Java program to demonstrate exception handling using try and catch blocks, nested and multiple try-catch blocks.

Problem Statement

Write a Java program to demonstrate exception handling using try and catch

blocks, nested and multiple try-catch blocks.

Learning Objectives

To handle runtime exceptions using try-catch, nested try, and multiple catch blocks.

Theory

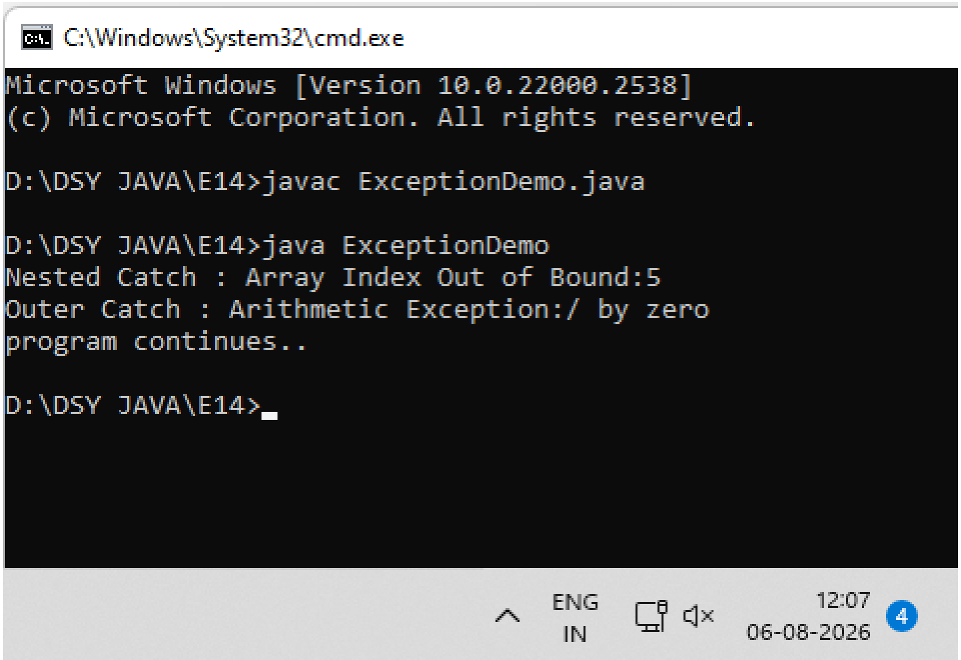
Exception handling prevents abnormal termination of programs due to runtime errors. The try block contains code that may generate exceptions, while catch blocks handle specific exceptions. Nested and multiple try-catch structures allow complex error management. Exception handling improves software reliability and user experience by gracefully managing unexpected situations.

Code

```
import java.io.*;

public class ExceptionDemo{
  public static void main(String []a){
    try{
      int arr[]={10,20,30,40};
      try{

        try{

        int result=arr[5];
        System.out.println(result);
      }catch(ArrayIndexOutOfBoundsException e){

        System.out.println("Nested Catch : Array Index Out of Bound:"+e.getMessage());
```

```
        }

        int num=10/0;
        System.out.println(num);

        }catch(ArithmeticException e){
        System.out.println("Outer Catch : Arithmetic Exception:"+e.getMessage());
        }

        }catch(Exception e){
        System.out.println("General Exception:"+e.getMessage());
      }

      System.out.println("program continues..");
    }
}
```

Output



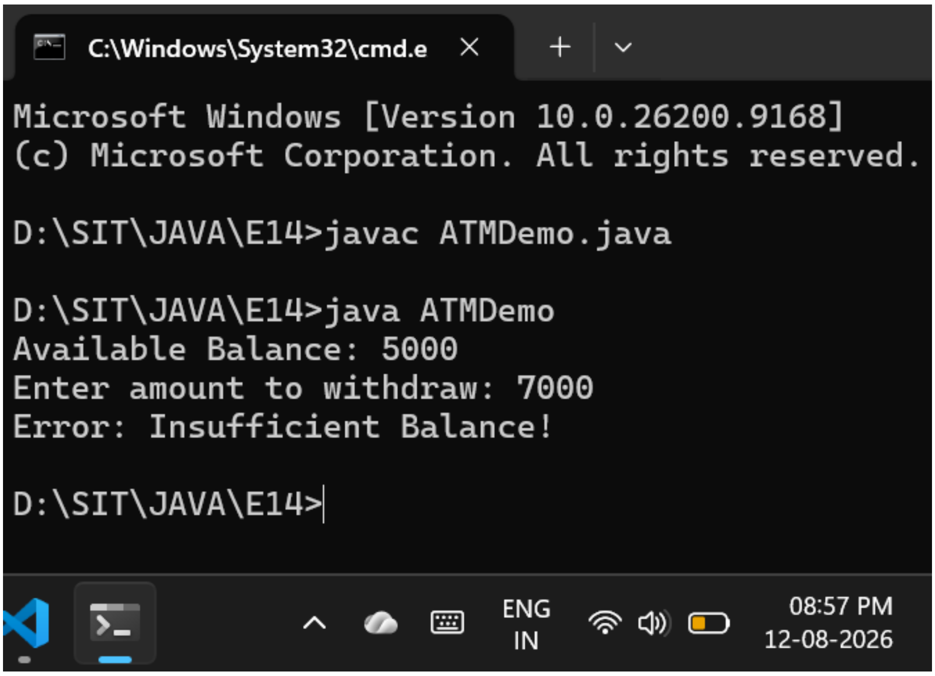
Exercise

1. Create an ATM program handling invalid withdrawal amount using try-catch

Code

```
import java.io.*;
import java.util.Scanner;

public class ATMDemo {
  public static void main(String[] args) {
    int balance = 5000;
    Scanner scanner = new Scanner(System.in);

    System.out.println("Available Balance: " + balance);
    System.out.print("Enter amount to withdraw: ");

    try {
      int amount = scanner.nextInt();

      if (amount > balance) {
        throw new IllegalArgumentException("Insufficient Balance!");
      } else if (amount <= 0) {
        throw new IllegalArgumentException("Amount must be greater than zero!");
      }

      balance = balance - amount;
      System.out.println("Please collect your cash.");
      System.out.println("Remaining Balance: " + balance);

    } catch (IllegalArgumentException e) {
      // Catches the invalid amount error
      System.out.println("Error: " + e.getMessage());

    } catch (Exception e) {
      System.out.println("Error: Invalid input! Please enter a number.");
    }

    scanner.close();
  }
}
```

Output



2. Create an Online Shopping program that handles an invalid product quantity entered by the user using a try-catch block. Display an appropriate error message if the quantity is less than or equal to zero.

Code

```
import java.io.*;

public class OnlineShopping {

  void processOrder(int quantity) {
    int pricePerItem = 500;

    if (quantity <= 0) {
      throw new IllegalArgumentException("Invalid Quantity! Quantity must be 1 or more.");
    }

    int totalCost = quantity * pricePerItem;
    System.out.println("Success! You ordered " + quantity + " items. Total: " + totalCost);
  }

  public static void main(String[] args) {
    OnlineShopping s = new OnlineShopping();
```

```
    try {
      System.out.println("\nfirst Order");
      s.processOrder(2);
    } catch (IllegalArgumentException e) {
      System.out.println("Error: " + e.getMessage());
    }

    System.out.println("\nSecond Order");
    try {

      s.processOrder(0);
    } catch (IllegalArgumentException e) {
      System.out.println("Error Caught: " + e.getMessage());
    }

    System.out.println("\nProgram ended.");
  }
}
```

Output